ZOKA — Archived Whitepaper v1.0

> **Historical document — correction added 11 October 2026.** The original version 1.0 text below is retained as a historical record. It describes Zenith, now designated for testnet use, and is superseded by the Quantic v2 design draft. Its former mainnet labels, roadmap, downloads and privacy claims are not current guidance.

> **Privacy correction:** Zenith publishes the commitments and one-time public keys of consumed notes. This exposes spend linkage even without a quantum computer. Claims below that ring signatures hide the real input, that input/output links are hidden, or that the anonymity formulas quantify the shipped protocol are withdrawn. Linkage does not by itself reveal a person's identity or every amount, but off-chain information can identify users.

> **Quantum correction:** the statements below that past privacy survives a quantum break are incorrect. An adversary with the necessary quantum capability and a recipient's address could attack archived classical key agreement and encrypted payment data.

> **Economics correction:** "no premine" applies to historical Zenith only. Proposed Quantic supply is 23,000,000 ZKA, including a 5,000,000 ZKA team premine spendable at block 0 without a lock-up; the remaining 18,000,000 ZKA are mined. Zenith balances do not migrate. Public block subsidies also reveal issuance amounts despite private recipients.

Original Version 1.0 Text

Privacy by Default Blockchain — Version 1.0

Status: Mainnet Zenith — public, permissionless, mining open.

Abstract

ZOKA is a privacy-first blockchain that hides sender, recipient and amount of every value-bearing transaction by construction. There is no transparent mode, no opt-in shield: the protocol refuses to broadcast economic data in cleartext.

The implementation combines five well-understood cryptographic primitives:

1. **Groth16 zk-SNARKs** over BLS12-381 for transaction validity proofs.
2. **Pedersen commitments** over Ristretto255 for confidential amounts.
3. **Bulletproof range proofs** to prevent inflation under hidden amounts.
4. **CLSAG ring signatures** to hide the sender inside a candidate ring.
5. **Stealth addresses** with separated scan and spend keys so the same recipient never appears twice on chain.

Consensus is **RandomX-compatible Proof of Work** (`zoka-randomx-v1`). Block production is permissionless, CPU-friendly, and ASIC-resistant. The native token is **ZKA**, capped at 23,000,000 units, with no premine

and no foundation allocation.

This document describes the protocol that the Mainnet Zenith implementation actually ships. Aspirational features are marked **Planned** and not claimed as live.

---

1. Scope and Status

This whitepaper describes Version 1.0 of the ZOKA protocol as implemented in the public reference codebase at github.com/ZokaNetwork/ZokaNetwork.

Live in Mainnet Zenith:

- Privacy-by-default private transactions (ring sigs + stealth + Pedersen + Bulletproofs + Groth16).
- RandomX-compatible Proof of Work consensus.
- libp2p peer-to-peer transport with gossipsub.
- Public bootstrap node and signed mainnet manifest.
- Open-source `zoka` CLI and `zokahd` daemon.
- Cross-platform desktop wallet (ZSilent Core v1.1.2, Windows and Linux).
- Android wallet (ZKAPriv v1.0.0 APK).

Planned, not yet shipped:

- External cryptographic audit. The code has been reviewed internally; an independent audit is required before this document can be treated as a formally verified specification.
- Tor / I2P anonymous transport. The mainnet manifest already declares the policy, but the runtime currently uses clear TCP.
- macOS and iOS clients.
- Multi-party computation (MPC) ceremony for a fresh trusted setup. The current Groth16 parameters in `trusted-setup/` ship with a DIGEST file so that every node can verify the hash matches the network-expected value; a public MPC remains future work.

No claim is made here that the planned milestones are already complete.

---

2. Threat Model

The protocol is designed to defend against:

1. A passive observer of the public chain, including any third party indexing block data.
2. A node operator who can read all gossiped traffic on the libp2p mesh.
3. An adversary who obtains the encrypted wallet file but does not have the wallet password.
4. A miner who attempts to censor or front-run transactions (MEV-style attacks).

The protocol does **not** defend against:

1. A global passive adversary observing both ends of every network link in real time.

2. Post-quantum adversaries. BLS12-381 and the discrete-log assumptions underpinning ring signatures and stealth addresses are not post-quantum safe today. Past privacy survives a future quantum break; new transactions issued after such a break would not.
3. A user who reuses zka1 receive addresses across distinct off-chain identities.
4. A user with a compromised spending key. There is no recovery backdoor.
5. Local disk compromise combined with knowledge of the wallet password.

---

3. Cryptographic Stack

3.1 Curves and Domains

Curve	Use
BLS12-381	SNARK proving and verification.
Ristretto255	Ring signatures, Pedersen commitments, stealth keys, range proof commitments.

3.2 Pedersen Commitments

For a value `v` and a blinding factor `r`:

```
C(v, r) = v*G + r*H
```

with independent generators `G` and `H` over Ristretto255. Properties:

- Perfectly hiding.
- Computationally binding.
- Additively homomorphic, so `C(v1) + C(v2) = C(v1 + v2)`.

3.3 Confidential Balance Equation

Every private transaction satisfies:

```
sum(C_in) = sum(C_out) + C_fee + X*H
```

where `C_fee = fee*G + r_fee*H` and `X = sum(r_in) - sum(r_out) - r_fee`. Validation fails if this equation does not hold over the curve group, which guarantees inputs equal outputs without the network ever seeing a cleartext amount.

3.4 Range Proofs

Each committed amount is constrained to:

```
0 <= v < 2^32
```

via Bulletproof verification. This prevents overflow-based inflation while keeping the value encrypted. Typical proof size is roughly 600 bytes.

3.5 Nullifiers

Each consumed note yields a single-use nullifier:

```
N = H(s || idx || nonce)
```

The network maintains a set of seen nullifiers and rejects any transaction whose inputs would re-emit one. Double-spend prevention is therefore enforced without any link between the nullifier and a specific output.

3.6 CLSAG Ring Signatures

For ring size `N`, the signature payload layout is:

```
sig = c0 || key_image || s0 || s1 || ... || s_(N-1)
```

Serialized size:

```
L_sig(N) = 32 * (N + 2) bytes
```

Verification is linear in `N`. The implementation lives at `src/private_wallet/clsag.rs` and is exercised by the test vectors in `src/private_wallet/clsag_vectors.json`.

3.7 Stealth Addresses

Recipients hold two key pairs: `(s, S=sG)` for scanning and `(b, B=bG)` for spending. The sender computes a one-time output public key:

```
P = H(r*S || idx)*G + B
```

where `r` is the sender's per-output randomness. The recipient scans the chain with `s` and recognises outputs destined to them; only the holder of `b` can spend them. Reusing the recipient identity does not produce repeated on-chain addresses.

---

4. Transaction and Validation Pipeline

A private transaction carries:

- Ring metadata: `ring_members`, `ring_signature`, `ring_version`.
- Input nullifiers and input commitments.
- Output commitments and range proofs.
- Fee commitment and fee range proof.
- Groth16 SNARK proof and Merkle root anchors.

Validation sequence (executed by every full node):

1. Structural and version checks.
2. Nullifier uniqueness against the current nullifier set.
3. Ring signature verification.
4. Range proof verification for every output and the fee.
5. Commitment balance equation over the group.
6. Inclusion proof check that ring members and notes exist in the chain.
7. Groth16 SNARK verification with the trusted-setup verifying key.

`ring_version = 1` is the active runtime path for private ring validation. Any transaction that fails one of these checks is rejected at mempool time and never reaches the chain.

---

5. Consensus: RandomX Proof of Work

5.1 Algorithm

ZOKA uses a RandomX-compatible PoW algorithm identified by the network as `zoka-randomx-v1`. RandomX is a CPU-friendly proof of work designed to be ASIC-resistant by executing random programs over a large dataset that fits in CPU caches and DRAM.

5.2 Block Cadence

Parameter	Value
Target block time	120 seconds
Coinbase maturity	60 blocks
Mining key rotation interval	2,048 blocks (~2.8 d)
Difficulty adjustment	Per block, bounded

Finality is probabilistic. The protocol does not implement deterministic finality (PBFT or similar). Applications requiring strong settlement should wait for several confirmations; twelve is a conservative default.

5.3 Mining Participation

Mining is permissionless. Any operator can run the node with a private mining address:

```
zoka fullnode --network mainnet \
 --address zka1yourshieldedaddress... \
 --threads <N>
```

The mining reward is emitted as a private coinbase output. Observers see that a block was mined; the chain never reveals how much the miner earned or which address received it.

5.4 No Validators, No Stake

There is no validator set, no proof of stake mechanism, and no economic finality. Mining participation is bounded only by computational resources and electricity. The `src/consensus/pbft.rs` module in the codebase is unused in the mainnet runtime and is retained as scaffolding for future research; the mainnet runtime path is PoW only.

---

6. Token Economics — ZKA

Parameter	Value
Symbol	ZKA
Decimals	6
Smallest unit	1 atom = 10^{-6} ZKA
Max supply	23,000,000 ZKA = $23 \cdot 10^{12}$ atoms
Genesis supply	0 ZKA
Initial reward	12 ZKA / block
Halving interval	1,051,200 blocks (~4 years)
Tail emission	None

Emission schedule:

Era	Reward	End block	Cumulative supply
0	12 ZKA	1,051,200	~12.6 M ZKA
1	6 ZKA	2,102,400	~18.9 M ZKA
2	3 ZKA	3,153,600	~22.0 M ZKA
3	1 ZKA	exhausts cap	23 M ZKA

Emission stops at the 23 M cap. There is no premine, no founder allocation, no foundation treasury. Every ZKA in circulation has been produced by a public block reward. Parameters are verifiable in `src/economics.rs` and `docs/economics.testnet.v1.json` of the network repository.

---

7. Network Architecture

7.1 Peer-to-peer Transport

Nodes communicate over libp2p. The protocol stack:

- TCP listener for inbound peer connections (default port 30303).

- Gossipsub for block and transaction propagation.
- Identify + ping for peer health.
- A protocol freeze identifier (`/zoka/1.0.0`) ensures peers refuse to interoperate across incompatible protocol versions.

The mainnet manifest at `docs/mainnet-network.v1.json` lists the public seed node, the read-only RPC endpoint, and the transport policy. It is signed by the network operators and shipped inside every binary release via `include_str!`.

7.2 Public Bootstrap

A read-only Mainnet Zenith bootstrap node runs at:

```
do-nyc-bootstrap-1
/ip4/159.223.119.216/tcp/28080
```

The active public HTTP RPC is `http://159.223.119.216:3000`; the fallback RPC is `http://157.230.5.103:3000`. These endpoints serve discovery and initial sync. They do not hold user secrets, broadcast private keys, or arbitrate the chain.

7.3 Embedded Trusted Setup

The Groth16 proving and verifying parameters live in the network repository at `trusted-setup/`. Files:

File	Purpose
`balance.params`	Balance circuit parameters.
`merkle.params`	Merkle membership circuit.
`nullifier.params`	Nullifier circuit.
`private_tx.params`	Combined private-tx circuit.
`DIGEST`	SHA-256 of the parameter bundle.

Every node enforces that the local trusted-setup directory's digest matches the expected network digest (`ZOKA_EXPECTED_SETUP_DIGEST`). A mismatch refuses to start the node. This guards against an operator silently substituting a different CRS.

7.4 Binaries

The network repository builds two binaries:

Binary	Role
`zoka`	CLI for wallet ops, sync gating, mining control, full-node orchestration.
`zokahd`	Headless daemon variant intended for server / VPS deployment.

Both binaries are bundled inside the ZSilent Core desktop wallet and supervised by it; the desktop wallet is therefore not a separate client implementation but a UI over the same canonical node software.

---

8. Privacy Guarantees

8.1 What the Chain Hides

Field	Mechanism
Sender	CLSAG ring signature
Recipient	Stealth address (one-time output key)
Amount	Pedersen commitment
Fee value	Pedersen commitment
Coinbase reward value	Pedersen commitment
Link between inputs and outputs	Groth16 proof of validity

8.2 What the Chain Necessarily Publishes

Field	Reason
Block height and timestamp	Required for consensus.
Transaction hash	Required for chain referencing.
Fact a transaction occurred	Inherent to a public ledger.
Nullifiers	Required for double-spend prevention; not linkable to outputs.
Merkle / state roots	Required for sync and inclusion proofs.
PoW difficulty, block size, gas-style budget metadata	Required for validation.

---

9. Anonymity Functions

The following functions quantify privacy under stated assumptions.

9.1 Sender Anonymity

Let `N` be the ring size and `q ∈ (0,1]` the decoy quality factor (1 means ideal decoys indistinguishable from the true spender). The effective ring size is:

```
N_eff = 1 + q * (N - 1)
```

The best posterior guess for the sender is:

```
P_sender = 1 / N_eff
```

The sender entropy in bits:

```
H_sender = log2(N_eff)
```

9.2 Network Source Exposure (Planned Tor/I2P)

Let `c` be the fraction of adversarial first-hop candidates and `h` the stem hop count for Dandelion++ relay. The approximate stem compromise probability is:

```
P_stem_comp(c, h) = 1 - (1 - c)^h
```

The network anonymity score is:

```
A_net = 1 - P_stem_comp
```

This function describes the planned anonymous transport. Mainnet Zenith uses cleartext TCP; the value of `A_net` is therefore not currently bounded by this expression.

9.3 Timing Linkability

Let `b` be the effective batch cardinality observed by an external party. If the observer cannot order the sender inside the batch:

```
P_time_link = 1 / b
H_time      = log2(b)
```

9.4 Composite Privacy Index

Define normalised components in `[0, 1]`:

```
S = H_sender / H_ref       # sender entropy normalised to a reference
T = H_time   / H_ref       # timing entropy normalised
A = A_net                  # network anonymity (1.0 when Tor/I2P is live)

PI = (S + T + A) / 3
```

`PI = 1.0` represents the strongest privacy state achievable by the protocol's combined mechanisms. Mainnet Zenith typically operates at `S = H_sender / H_ref` with `A` reduced because Tor/I2P is still on the roadmap; users requiring maximum network anonymity should operate the node behind their own Tor circuit until the runtime ships native support.

---

10. Honest Limits

These are limits of the design, not bugs to be fixed:

- A global passive adversary observing both ends of every link can correlate burst timing. Tor and I2P, once shipped, mitigate but do not eliminate this risk.
- The current cryptographic stack is not post-quantum. A future quantum break would not retroactively reveal already-issued transactions, but it would compromise new ones.
- A compromised spending key has no recovery backdoor. Custody is yours, with everything that implies. A lost seed is lost funds.
- Reusing zka1 receive addresses across distinct off-chain identities can leak associations between them.
- The network has no involuntary unmasking path. Only the user holding the key can reveal a transaction. This is the design; the cost is that there is no support team that can recover funds.
- Block time is 120 seconds. The first confirmation is not final; settlement systems should wait several blocks.
- The trusted setup parameters in Mainnet Zenith are bootstrap parameters with a single-operator origin. A multi-party MPC ceremony to produce successor parameters is on the roadmap and the protocol is built to allow CRS upgrades without changing the chain history.

---

11. Implementation Status and Roadmap

Milestone	Status
Internal devnet	Done
Public testnet	Done
Mainnet Zenith (public bootstrap, mining open)	Live
ZSilent Core v1.1.2 desktop wallet (Windows / Linux)	Live
ZKAPriv v1.0.0 Android wallet	Live
ZSilent Core desktop wallet (macOS)	Planned
Public block explorer (zokaexplorer)	Live
Native Tor / I2P transport	Planned
External cryptographic audit	Planned
Multi-party trusted-setup ceremony	Planned
iOS mobile client	Planned
Post-quantum migration	Research

---

12. Reference Code and Reproducibility

Component	Repository
Network (Rust)	github.com/ZokaNetwork/ZokaNetwork
Desktop wallet	github.com/ZokaNetwork/zsilent-core
Android wallet	github.com/ZokaNetwork/zkapriv
Block explorer	github.com/ZokaNetwork/zokaexplorer
Marketing site	github.com/ZokaNetwork/zokaweb

Every released binary can be reproduced from the corresponding tagged source with the build workflow at `.github/workflows/release.yml`. Each release ships SHA-256 checksums and the trusted-setup DIGEST so an operator can verify, before running anything, that the local parameters match the network-expected hash.

---

13. Closing Position

ZOKA is the protocol I built because I believe the right to transact in private is the last frontier of individual freedom. It will not protect you from every threat; it does not promise perfection. What it promises is the strongest privacy this implementation knows how to ship, with the limits stated in plain text — not buried in a footnote.

Verify what is true. Reveal nothing else.

— Natosh Zoka, core dev @ ZokaNetwork